ĐẠI HỌC QUỐC GIA TPHCM

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

AI23 (23TNT1), FIT@HCMUS, VNUHCM

Đồ án 2 - Hashiwokakero

Đề tài: Xây dựng bộ giải Hashiwokakero sử dụng logic dạng chuẩn hội

Môn học: Cơ sở trí tuệ nhân tạo - TTNT2023

Sinh viên thực hiện:
Đinh Đức Anh Khoa (23122001)
Nguyễn Lê Hoàng Trung (23122004)
Đinh Đức Tài (23122013)

Giáo viên hướng dẫn:
GS.TS Lê Hoài Bắc
ThS. Lê Nhựt Nam

Ngày 30 tháng 12 năm 2025

Mục lục

1 Giới thiệu **2**

2 Hashiwokakero **3**

3 Ràng buộc CNF và PySAT cho Hashiwokakero **3**
3.1 Biểu diễn bài toán . . . 3
3.1.1 Định nghĩa biến logic . . . 3
3.1.2 Ràng buộc CNF . . . 4
3.2 Giải bài toán thỏa mãn Boolean bằng PySAT . . . 5
3.3 Sinh bản đồ Hashiwokakero . . . 5

4 A*, Brute-force và Backtracking cho Hashiwokakero **6**
4.1 Thuật toán Brute-force . . . 6
4.2 Thuật toán Backtracking . . . 6
4.3 Thuật toán A* . . . 7

5 Thực nghiệm **9**
5.1 Phương pháp . . . 9
5.1.1 Dữ liệu . . . 9
5.1.2 Quy trình thực nghiệm . . . 10
5.2 Kết quả thực nghiệm . . . 10

6 Kết luận, đánh giá **11**
6.1 Kết luận . . . 11
6.2 Đánh giá . . . 12

1 Giới thiệu

Đây là tài liệu báo cáo cho **Đồ án 2 - Hashiwokakero**, môn **Cơ sở trí tuệ nhân tạo - TTNT2023**, Khoa Công nghệ thông tin, Trường Đại học Khoa học tự nhiên - Đại học Quốc gia TP.HCM. Chủ đề của đồ án là phát triển bộ giải câu đố Hashiwokakero sử dụng logic dạng chuẩn hội (CNF).

Báo cáo được thực hiện bởi nhóm các thành viên:

- Đinh Đức Anh Khoa (23122001)
- Nguyễn Lê Hoàng Trung (23122004)
- Đinh Đức Tài (23122013)

Đường dẫn repository Github: https://github.com/anhKhoa585/AI-Fundamentals-Proj2 [1]

Demo youtube: https://youtu.be/rRrwQ48saS8 [2]

Bảng phân công nhiệm vụ cho từng thành viên:

Họ và tên	MSSV	Nhiệm vụ
Đinh Đức Anh Khoa	23122001	- Chạy so sánh, thực nghiệm, trực quan hóa - Quay video demo - Tổ chức và cải tiến mã nguồn.
Nguyễn Lê Hoàng Trung	23122004	- Báo cáo và code thuật toán Brute-force, Backtracking, A* giải CNF cho bài toán Hashiwokakero
Đinh Đức Tài	23122013	- Phát biểu bài toán Hashiwokakero - Định nghĩa biến logic, ràng buộc CNF - Code bộ tạo Hashi, tự động sinh CNF - Phát biểu và Code bộ giải Hashi bằng PySAT (Glucose 3)

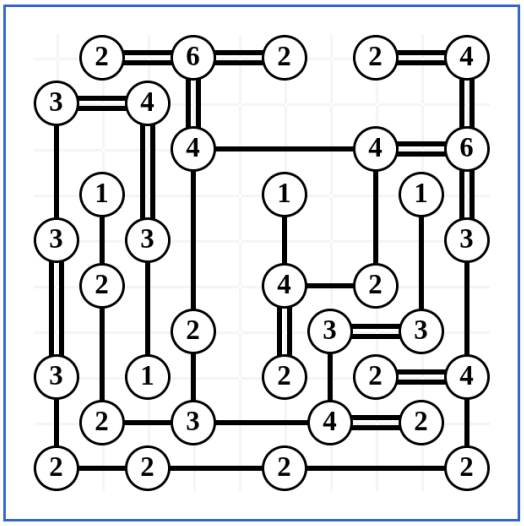
Các công nghệ sử dụng:

- **Python**: Python là một ngôn ngữ lập trình bậc cao, đa năng. Python được sử dụng rộng rãi trong các lĩnh vực như phát triển web, khoa học dữ liệu, trí tuệ nhân tạo, và tự động hóa.
- **pySAT**: pySAT là một bộ công cụ dựa trên Python nhằm mục đích nghiên cứu và phát triển các bộ giải quyết Bài toán thỏa mãn Boolean (Boolean Satisfiability Problem - SAT)

2 Hashiwokakero

Hashiwokakero (xây cầu) là một loại trò chơi giải đố do Nikoli xuất bản. Cho một tập hợp các điểm trên mặt phẳng (đảo), người chơi phải kết nối chúng bằng các đoạn thẳng dọc và ngang (cầu). Mỗi đảo cho trước số lượng cầu cần kết nối với nó. Một cây cầu không được cắt ngang bất kỳ cây cầu hoặc đảo nào khác và tối đa hai cây cầu được chạy song song. Tất cả các đảo phải tạo thành một hệ thống liên thông.

Hashiwokakero được chứng minh là một bài toán thuộc lớp NP-đầy đủ [3], tức là không có thuật toán nào có thể giải hiệu quả mọi màn chơi Hashiwokakero khi kích thước bàn chơi tăng lên.



Hình 1: Một màn chơi Hashiwokakero với kích thước bảng 10x10

3 Ràng buộc CNF và PySAT cho Hashiwokakero

3.1 Biểu diễn bài toán

Ta có thể mô hình hóa Hashiwokakero dưới dạng các biến logic và ràng buộc CNF.

3.1.1 Định nghĩa biến logic

Đầu tiên, xác định tất cả các cặp đảo có thể nối với nhau. Hai đảo nối được với nhau nếu chúng cùng hàng/cột và không có đảo nào chắn giữa. Một cạnh nối hai đảo `(u, v)` được đưa vào danh sách $E_{potential}$ (cạnh tiềm năng) nếu `u, v` nằm cùng hàng/cột và không có đảo nào khác nằm chắn giữa. Vì giữa 2 đảo `(u, v)` có thể có 0, 1 hoặc 2 cây cầu, nên với mỗi cạnh tiềm năng nối đảo `u` và đảo `v`, ta định nghĩa 2 biến logic:

- $x_{uv,1}$: Có ít nhất 1 cây cầu giữa `u` và `v`.
- $x_{uv,2}$: Có ít nhất 2 cây cầu giữa `u` và `v` (có đúng 2 cầu).

Chân trị của $x_{uv,1}$ và $x_{uv,2}$ ứng với số cầu được biểu diễn như sau:

- 0 cầu: $x_{uv,1} = false$ và $x_{uv,2} = false$.
- 1 cầu: $x_{uv,1} = true$ và $x_{uv,2} = false$.
- 2 cầu: $x_{uv,1} = true$ và $x_{uv,2} = true$.

3.1.2 Ràng buộc CNF

Vì Hashiwokakero có các ràng buộc nghiêm ngặt về điều kiện dựng cầu, ta biểu diễn chúng dưới dạng logic và chuyển về dạng hội chuẩn CNF (Conjunctive normal form) như sau:

Ràng buộc nhất quán Nếu có 2 cầu thì chắc chắn phải có 1 cầu.

- Logic: $x_{uv,2} \to x_{uv,1}$
- CNF:

$$\neg x_{uv,2} \vee x_{uv,1} \tag{1}$$

Ràng buộc cầu cắt nhau Cầu ngang và cầu dọc không được cắt nhau. Giả sử cạnh ngang `(u, v)` cắt cạnh dọc `(a, b)`:

- Logic: Không thể đồng thời có cầu ở `(u, v)` và `(a, b)`.
- CNF:

$$\neg x_{uv,1} \vee \neg x_{ab,1} \tag{2}$$

Ràng buộc số lượng cầu nối với đảo Với mỗi đảo `u`, tổng số cầu nối phải nó phải bằng số K_u cho trước. Gọi tập các cạnh nối vào `u` là $Adj(u)$:

- Logic: $\sum_{e \in Adj(u)} (x_{e,1} + x_{e,2}) = K_u$
- CNF: Gọi $X_u = \{x_1, x_2, \ldots, x_{N_u}\}$ ($N_u = 2\,|Adj(u)|$) là tập hợp tất cả các biến logic (cả 1 cầu và 2 cầu) nối vào đảo `u`, ta xây dựng 2 ràng buộc:
  - Ràng buộc tối đa K_u (Không tồn tại bất kỳ tập con nào gồm K+1 biến mà tất cả đều bằng True):

$$\bigwedge_{S \subseteq X_u, |S| = K_u + 1} \left(\bigvee_{x \in S} \neg x \right) \tag{3}$$

  - Ràng buộc tối thiểu K_u (Không tồn tại bất kỳ tập con nào gồm N-K+1 biến mà tất cả đều bằng False):

$$\bigwedge_{Q \subseteq X_u, |S| = N_u - K_u + 1} \left(\bigvee_{x \in Q} x \right) \tag{4}$$

Ràng buộc đồ thị liên thông Để đảm bảo tất cả các đảo tạo thành một thành phần liên thông duy nhất, ta áp dụng phương pháp *loại bỏ tập con*. Do việc liệt kê tất cả các tập con ngay từ đầu là bất khả thi (độ phức tạp hàm mũ), ta sử dụng phương pháp *sinh ràng buộc trễ* với quy trình như sau:

- Bước 1: Gọi bộ giải (SAT, A*, Brute-force...) tìm một nghiệm M thỏa mãn các ràng buộc về nhất quán, cắt nhau và số lượng cầu.
- Bước 2: Dựng đồ thị G' dựa trên nghiệm M. Kiểm tra tính liên thông của G'.
- Bước 3: Nếu G' đã liên thông, M là nghiệm cuối cùng.
- Bước 4: Nếu G' không liên thông, giả sử G' có một thành phần liên thông con $S \subset V$ (với $0 < |S| < |V|$). Để kết nối S với phần còn lại của đồ thị, bắt buộc phải có ít nhất một cây cầu nối giữa một đảo thuộc S và một đảo không thuộc S. Ta thêm ràng buộc mới (gọi là *cut constraint*) vào bộ CNF:

$$\bigvee_{u \in S, v \notin S, (u,v) \in E_{potential}} x_{uv,1} \tag{5}$$

- Bước 5: Quay lại Bước 1 với tập ràng buộc đã được cập nhật.

Phương pháp này đảm bảo rằng khi thuật toán dừng lại, nghiệm tìm được chắc chắn thỏa mãn tính liên thông mà không cần sinh ra quá nhiều biến phụ trợ.

3.2 Giải bài toán thỏa mãn Boolean bằng PySAT

`Glucose` [4] là một trong những bộ giải SAT dựa trên phương pháp CDCL (Conflict-Driven Clause Learning - Học mệnh đề dựa trên xung đột) thành công và được sử dụng rộng rãi nhất. `Glucose` được xây dựng dựa trên mã nguồn của bộ giải `Minisat`, nhưng được cải tiến mạnh mẽ với các thuật toán quản lý mệnh đề mới.

Sau khi mô hình hóa Hashiwokakero thành các biến logic $x_{uv,1}$, $x_{uv,2}$ cùng với các ràng buộc, `Glucose` được sử dụng để tìm một phép gán thỏa mãn cho tất cả các ràng buộc này. Điểm mạnh của `Glucose` thể hiện rõ ở khả năng xử lý hiệu quả phương pháp *sinh ràng buộc trễ* cho điều kiện liên thông đồ thị: mỗi khi tìm được một nghiệm chưa liên thông, bộ giải chỉ cần thêm một ràng buộc *cut constraint* mới và tiếp tục tìm kiếm, thay vì phải liệt kê tất cả 2^n tập con ngay từ đầu.

3.3 Sinh bản đồ Hashiwokakero

Để phục vụ việc kiểm thử và minh họa, ta có một cách đơn giản để sinh bản đồ như sau:

- Bước 1: Khởi tạo bản đồ trống và đặt ngẫu nhiên 1 đảo đầu tiên.
- Bước 2: Chọn một đảo đã có, bắn một tia theo hướng ngẫu nhiên (lên, xuống, trái, phải) với độ dài ngẫu nhiên.
- Bước 3: Nếu điểm đến hợp lệ (không đè lên đảo khác, đường đi không cắt cầu khác), đặt một đảo mới. Nối 2 đảo bằng 1 hoặc 2 cây cầu, cộng số lượng cầu vào giá trị của 2 đảo. Lặp lại bước 2 cho đến khi số lượng đảo đạt mục tiêu hoặc số lần thử thất bại vượt quá giới hạn.

4 A*, Brute-force và Backtracking cho Hashiwokakero

4.1 Thuật toán Brute-force

Thuật toán Brute-force là phương pháp tìm kiếm liệt kê tất cả các trạng thái có thể có của các cạnh tiềm năng, sau đó kiểm tra xem trạng thái nào thỏa mãn toàn bộ tập ràng buộc CNF đã định nghĩa. Đây là phương pháp đơn giản nhất với quy trình như sau:

1. **Xác định không gian tìm kiếm:** Dựa trên tập cạnh tiềm năng $E_{potential}$, với mỗi $e \in E_{potential}$ có thể nhận một trong 3 trạng thái:
   - Không có cầu: Tương ứng biến logic $x_{e,1} = false$ và $x_e = false$
   - Có 1 cầu: Tương ứng biến logic $x_{e,1} = true$ và $x_{e,2} = false$
   - Có 2 cầu: Tương ứng biến logic $x_{e,1} = true$ và $x_{e,2} = true$

   Nếu có M cạnh tiềm năng, tổng số trạng thái cần kiểm tra là 3^M.
2. **Sinh trạng thái:** Thuật toán sử dụng bộ `generator` (của itertools) tạo ra toàn bộ mọi tổ hợp khả dĩ $true/false$ để sinh lần lượt từng trạng thái:
$$state = \{v_{e,k} \mid e \in E_{potential}, k \in \{1,2\} \text{ và } v_{e,k} \in \{true, false\}\} \tag{6}$$
3. **Kiểm tra ràng buộc:** Với mỗi trạng thái $state$ được sinh ra, thuật toán kiểm tra tính thỏa mãn của các ràng buộc:
   - Ràng buộc nhất quán.
   - Ràng buộc cầu cắt nhau.
   - Ràng buộc số lượng cầu nối với đảo.
   - Ràng buộc đồ thị liên thông.
4. **Điều kiện dừng:** Thuật toán dừng khi tồn tại một trạng thái thỏa mãn các ràng buộc logic trên, trả về lời giải tìm được.

Độ phức tạp thời gian là hàm mũ $O(3^M * P)$ (M là số cạnh tiềm năng, P là chi phí kiểm tra mỗi trạng thái), thuật toán rất chậm với các bàn cờ kích thước lớn. Vì thuật toán được xây dựng bằng cách "thử và sai" trên toàn bộ không gian mẫu, không được áp dụng các kỹ thuật suy diễn để loại bỏ sớm các nhánh không có nghiệm, nên hiệu năng của Brute-force rất thấp.

4.2 Thuật toán Backtracking

Thuật toán Backtracking là phương pháp tìm kiếm theo chiều sâu (Depth First Search - DFS) để khám phá không gian trạng thái của bài toán, xây dựng lời giải từng bước và quay lui ngay khi phát hiện một ràng buộc bị vi phạm. Đây là phương pháp nền tảng giúp tìm ra lời giải khi các kỹ thuật suy luận logic thông thường không thể xác định bước đi tiếp theo, với quy trình như sau:

1. **Chọn biến:** Thuật toán chọn tuần tự một cạnh tiềm năng e từ tập $E_{potential}$ chưa được gán giá trị.

2. **Phân nhánh:** Trên cạnh tiềm năng hiện tại, lần lượt thử gán các biến logic với giá trị $true/false$, ta có 3 trạng thái:
   - Không có cầu: Tương ứng biến logic $x_{e,1} = false$ và $x_{e,2} = false$)
   - Có 1 cầu: Tương ứng biến logic $x_{e,1} = true$ và $x_{e,2} = false$)
   - Có 2 cầu: Tương ứng biến logic $x_{e,1} = true$ và $x_{e,2} = true$)
3. **Kiểm tra hợp lệ:** Với mỗi trạng thái được gán, kiểm tra các ràng buộc logic:
   - Ràng buộc nhất quán.
   - Ràng buộc cầu cắt nhau.
   - Ràng buộc số lượng cầu nối với đảo.
4. **Đệ quy:**
   - Nếu phép gán thỏa các ràng buộc, gọi đệ quy quay lại Bước 1 để xử lý cạnh tiềm năng tiếp theo.
   - Nếu phép gán dẫn đến vi phạm ràng buộc, hủy bỏ phép gán và thử trạng thái tiếp theo.
5. **Điều kiện dừng:** Thuật toán dừng khi tất cả các cạnh đã được xét. Thực hiện kiểm tra ràng buộc đồ thị liên thông:
   - Nếu đồ thị liên thông, trả về lời giải tìm được.
   - Ngược lại, quay lui về trạng thái trước đó và tiếp tục đệ quy.

Thuật toán Backtracking đảm bảo tìm ra nghiệm, cài đặt đơn giản, không yêu cầu cấu trúc dữ liệu phức tạp. Vì được áp dụng kỹ thuật suy diễn để loại bỏ sớm các nhánh không có nghiệm, nên hiệu năng của thuật toán Backtracking có phần vượt trội hơn thuật toán Brute-force. Tuy nhiên, với độ phức tạp thời gian là hàm mũ $O(3^M * P)$ (M là số cạnh tiềm năng, P là chi phí kiểm tra mỗi trạng thái) thuật toán vẫn rất chậm với các bàn cờ kích thước lớn.

4.3 Thuật toán A*

Xét bài toán Hashiwokakero, thuật toán A* kết hợp với các ràng buộc logic để xây dựng các trạng thái và tìm lời giải. Thay vì Brute-force, thuật toán sử dụng các hàm đánh giá (f, g, h) và định nghĩa hàm heuristic phù hợp với bài toán để định hướng tìm kiếm lời giải nhanh chóng. Quy trình sử dụng thuật toán A* giải bài toàn Hashiwokakero như sau:

1. **Khởi tạo các tham số**:
   - Chuyển đổi bài toán sang dạng CNF, sử dụng các biến logic và ràng buộc CNF.
   - Xây dựng các ánh xạ và chỉ mục giúp tra cứu nhanh giữa các biến logic, cạnh tiềm năng, đảo và các mệnh đề CNF.
2. **Biểu diễn trạng thái**: Mỗi trạng thái *state* trong không gian tìm kiếm của A* được biểu diễn bởi một tuple đại diện các giá trị của biến logic, trong đó:

- $state[i]$ = True: biến logic thứ $i+1$ được gán giá trị True
- $state[i]$ = False: biến logic thứ $i+1$ được gán giá trị False
- $state[i]$ = None: biến logic thứ $i+1$ chưa được gán giá trị

Trạng thái ban đầu: $state_0 = (\text{None}, \text{None}, \ldots, \text{None})$

3. **Hàm đánh giá**: A* sử dụng hàm $f(state) = g(state) + w * h(state)$ để đánh giá mỗi trạng thái.

   - **Hàm chi phí thực tế** $g(state)$: Là tổng số lượng cầu đã xây tại trạng thái $state$ hiện tại, được tính bởi công thức sau:

   $$g(state) = \sum_{e \in E_{potential}} (x_{e,1} + x_{e,2}), \text{ sao cho } x_{e,1}, x_{e,2} \in state \tag{7}$$

   Miền giá trị: $[0, 2 \times |E_{potential}|]$.

   - **Hàm heuristic** $h(state)$: Sử dụng mật độ nhu cầu để đo lường độ quan trọng trung bình của việc xây cầu với trạng thái $state$. Giá trị càng cao nghĩa là trạng thái này đang gặp khó khăn (thiếu nhiều cầu so với số cạnh còn lại), dẫn đến giá trị $f(state)$ cao. Do A* ưu tiên f nhỏ nhất, trạng thái này sẽ có độ ưu tiên thấp và được xếp lịch xử lý sau.

     (a) Số lượng cầu cần phải xây:

     $$missing = \sum_{u \in V} (K_u - \sum_{e \in Adj(u)} (x_{e,1} + x_{e,2})) \tag{8}$$

     với K_u là số cầu yêu cầu của đảo u, $Adj(u)$ là tập các cạnh nối vào u.

     (b) Số lượng cạnh tiềm năng chưa được xét:

     $$unassigned = \sum_{e \in E_{potential}} \mathbb{I}_{[x_{e,1} = \text{None}]} \tag{9}$$

     trong đó $\mathbb{I}$ là hàm chỉ thị, trả về 1 nếu điều kiện đúng.

     Công thức heuristic:

     $$h(state) = \begin{cases} \dfrac{missing}{unassigned} & \text{nếu } unassigned > 0 \\ 0 & \text{nếu } unassigned = 0 \end{cases} \tag{10}$$

     Miền giá trị: $[0, 2]$.

   - **Trọng số heuristic** w: Vì hàm heuristic sử dụng mật độ nhu cầu nên giá trị $h(state)$ rất nhỏ so với $g(state)$. Để cân bằng ảnh hưởng của heuristic trong hàm đánh giá, cần nhân $h(state)$ với trọng số:

     $$w = |E_{potential}| \tag{11}$$

     Khi đó $w \times h(state)$ có miền giá trị $[0, 2 \times |E_{potential}|]$, tương đồng với miền giá trị của $g(state)$.

4. **Khởi tạo OPEN List**: Sử dụng một hàng đợi ưu tiên để lưu các trạng thái chưa được khám phá, sắp xếp theo giá trị $f(state)$. Mỗi phần tử trong hàng đợi có dạng $(f, counter, state)$, trong đó *counter* dùng để phân định thứ tự khi hai trạng thái có cùng giá trị f. Thêm trạng thái khởi tạo $state_0$ vào OPEN list.

5. **Vòng lặp tìm kiếm**:

   - Lấy trạng thái *state* có $f(state)$ nhỏ nhất từ OPEN list.
   - Nếu *state* đã tồn tại trong CLOSED set, bỏ qua và lấy trạng thái tiếp theo.
   - Thêm *state* vào CLOSED set đánh dấu đã duyệt.
   - Nếu tất cả các biến trong *state* đã được gán giá trị, kiểm tra ràng buộc đồ thị liên thông. Nếu thỏa mãn, trả về *state* như một giải pháp.
   - Nếu không, chọn biến tiếp theo để gán dựa theo heuristic MRV (Minimum Remaining Values): ưu tiên đảo có tỉ lệ $\frac{missing}{remaining_edges}$ cao nhất.
   - Tạo các trạng thái con $state'$ bằng cách gán giá trị True hoặc False cho biến được chọn.
   - Với mỗi $state'$, kiểm tra các mệnh đề CNF liên quan đến biến vừa gán. Nếu không vi phạm ràng buộc, tính $f(state')$ và thêm vào OPEN list.

6. **Điều kiện dừng**: Thuật toán kết thúc khi:

   - Tìm thấy giải pháp: Trạng thái với tất cả biến đã gán và thỏa mãn ràng buộc liên thông.
   - OPEN list rỗng: Không còn trạng thái nào để khám phá, bài toán không có lời giải.

5 Thực nghiệm

5.1 Phương pháp

Để đánh giá hiệu quả của các thuật toán đã cài đặt (Brute-force, Backtracking, A*, và PySAT), nhóm thực hiện so sánh thời gian thực thi trên các bộ dữ liệu có kích thước tăng dần.

5.1.1 Dữ liệu

Dữ liệu đầu vào được sinh ngẫu nhiên thông qua bộ generator. Để đảm bảo tính khách quan, với mỗi kích thước bản đồ, nhóm thực hiện chạy **30** test case khác nhau. Cấu hình chi tiết các bộ dữ liệu như sau:

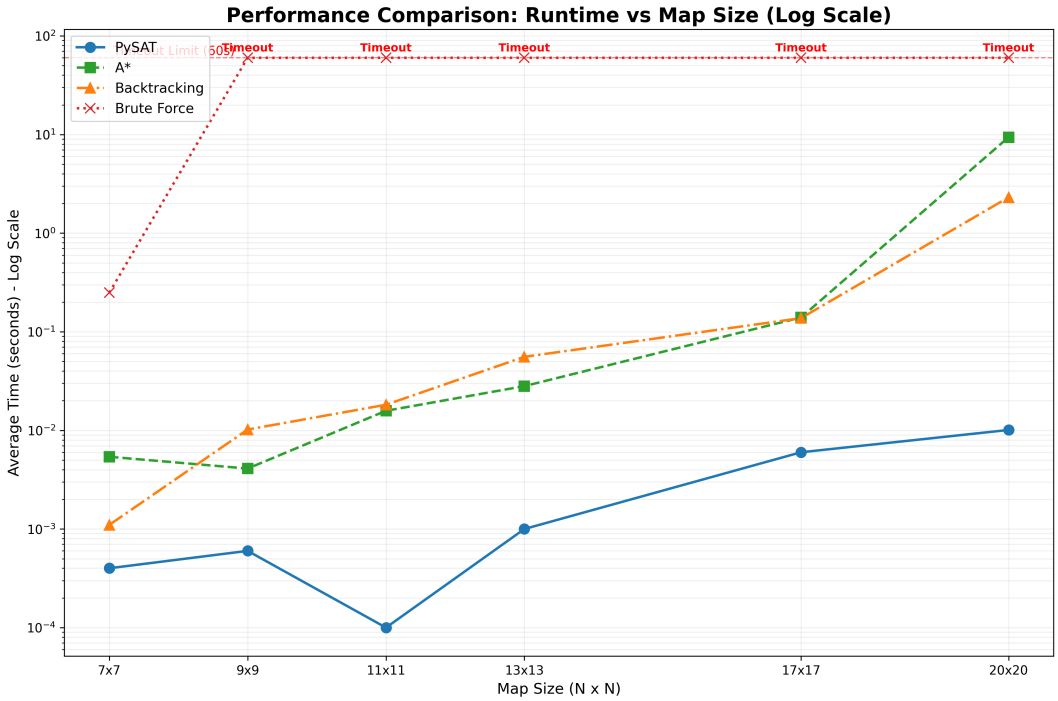
- **7x7:** 6 - 8 đảo.
- **9x9:** 10 - 12 đảo.
- **11x11:** 14 - 16 đảo.
- **13x13:** 18 - 22 đảo.
- **17x17:** 25 - 30 đảo.
- **20x20:** 40 - 50 đảo.

5.1.2 Quy trình thực nghiệm

1. **Thiết lập timeout:** Giới hạn thời gian chạy cho mỗi thuật toán trên một test case là **60 giây**.
2. **Đo lường:** Thời gian được tính từ lúc thuật toán bắt đầu giải đến khi tìm ra nghiệm hoặc bị ngắt do quá giờ.
3. **Xử lý timeout:** Nếu thuật toán chạy quá 60 giây, kết quả sẽ được ghi nhận là > 60 và gán giá trị là 60 giây để vẽ biểu đồ.
4. **Tính trung bình:** Kết quả cuối cùng là trung bình cộng thời gian của 5 lần chạy cho từng kích thước.

5.2 Kết quả thực nghiệm

Kết quả thực nghiệm được trực quan hóa bằng biểu đồ thang đo Logarithmic dưới đây để làm rõ sự chênh lệch lớn về hiệu năng giữa các thuật toán:



Hình 2: Biểu đồ so sánh thời gian chạy (Log Scale) theo kích thước bản đồ.

Quan sát biểu đồ hình 2, ta có những nhận xét chi tiết sau:

1. **Sự vượt trội của PySAT (Đường màu xanh lam):**
   - PySAT là thuật toán hiệu quả nhất.
   - Trên thang đo Log, đường biểu diễn của PySAT nằm thấp hơn các thuật toán khác từ 2 đến 3 bậc độ lớn.

- Điều này khẳng định sức mạnh của việc mô hình hóa bài toán về dạng CNF.

2. **Hiệu năng của Brute-force (Đường màu đỏ):**
   - Thuật toán Brute-force chỉ giải được kích thước nhỏ nhất (7x7).
   - Ngay khi kích thước tăng lên 9x9, thời gian chạy lập tức vượt quá giới hạn 60 giây do bùng nổ không gian trạng thái.
3. **So sánh giữa Backtracking và A*:**
   - Cả hai thuật toán đều giải quyết được tất cả các test case trong thời gian cho phép, tốt hơn nhiều so với Brute-force.
   - Ở các kích thước trung bình (9x9 đến 13x13), A* nhanh hơn Backtracking một chút nhờ hàm Heuristic định hướng tìm kiếm tốt.
   - Tuy nhiên, tại kích thước lớn nhất (20x20), A* lại trở nên chậm hơn Backtracking. Nguyên nhân có thể do chi phí quản lý hàng đợi ưu tiên và tính toán hàm Heuristic trong A* bắt đầu lớn hơn. trong khi đó Backtracking lại đơn giản và thực thi nhanh hơn.

Kết luận: Thực nghiệm cho thấy với bài toán Hashiwokakero, phương pháp giải bằng SAT (PySAT) là lựa chọn tối ưu nhất về tốc độ, khả năng mở rộng và tính ổn định.

6 Kết luận, đánh giá

6.1 Kết luận

Thông qua quá trình nghiên cứu và thực hiện đồ án giải quyết bài toán Hashiwokakero, nhóm đã đạt được những kết quả quan trọng sau:

- **Đa dạng hóa phương pháp tiếp cận:** Đồ án đã cài đặt và so sánh thành công 4 thuật toán khác nhau: Brute-force, Backtracking, A* và PySAT. Điều này giúp cung cấp cái nhìn toàn diện về ưu và nhược điểm của từng chiến lược khi giải quyết bài toán thỏa mãn ràng buộc (CSP).
- **Đánh giá hiệu năng:**
  - *Brute-force* Tuy dễ cài đặt nhưng chỉ hiệu quả với các bảng chơi kích thước nhỏ, gặp khó khăn lớn về thời gian khi không gian trạng thái mở rộng.
  - *A*:* Cải thiện đáng kể tốc độ tìm kiếm nhờ hàm heuristic định hướng, giảm bớt số lượng trạng thái cần duyệt.
  - *Backtracking*: Ý tưởng cài đặt đơn giản nhưng cho hiệu năng tương đương và có phần tốt hơn A*.
  - *PySAT:* Được chứng minh là phương pháp tối ưu và mạnh mẽ nhất trong đồ án, giải quyết tốt các bảng chơi lớn và phức tạp nhờ việc chuyển đổi bài toán về dạng CNF và tận dụng sức mạnh của các bộ giải SAT hiện đại.
- **Xây dựng hệ thống hoàn chỉnh:** Đã hoàn thiện quy trình từ việc sinh dữ liệu, mô hình hóa bài toán, giải quyết vấn đề đến việc chạy thực nghiệm để đưa ra các số liệu so sánh khách quan.

6.2 Đánh giá

Đối với đồ án này, nhóm tự đánh giá **hoàn thành tốt và đầy đủ** các yêu cầu đã được đặt ra, chi tiết như sau:

- Thực hiện đẩy đủ các bước gồm định nghĩa biến logic, xây dựng ràng buộc CNF, cài đặt thuật giải PySAT, A*, Backtracking, Brute-force cho bài toán Hashiwokakero.
- Có chạy thực nghiệm, so sánh, phân tích đầy đủ, chi tiết và làm video demo các thuật toán.

Tài liệu

[1] Anh Khoa, Hoang Trung, and Duc Tai. github/anhKhoa585/AI-Fundamentals-Proj2. https://github.com/anhKhoa585/AI-Fundamentals-Proj2, 2025.

[2] Anh Khoa, Hoang Trung, and Duc Tai. Demo Hashiwokakero solver using Conjunctive Normal Form. https://youtu.be/rRrwQ48saS8, 2025.

[3] Daniel Andersson. Hashiwokakero is np-complete. *Information Processing Letters*, 109(19):1145–1146, 2009.

[4] Gilles Audemard and Laurent Simon. On the Glucose SAT solver. *International Journal on Artificial Intelligence Tools (IJAIT)*, 27(1):1–25, 2018.